



Diseño de Bases de Datos: Arquitectura Híbrida para Ecommify

Grupo:

Andres Camilo Meneses Ortega
David Hernando Monsalve Delima
Eduardo Trujillo Santos
Nicolás Cifuentes Barriga

Maestría en Arquitectura de Software, Universidad de la Sabana

Diseño y Optimización de Bases de Datos MAS 2026-3 G1G2

Miguel Alfonso Varela

2026

1. Análisis de Principios de Normalización (1FN a 5FN)

La normalización es el proceso sistemático de organizar los datos de una base de datos relacional con el objetivo de reducir la redundancia y mejorar la integridad de los datos. A continuación se aplica este proceso al dataset de e-commerce Olist, partiendo de la tabla denormalizada OrdersFull.

1.1 Tabla Denormalizada: OrdersFull

El notebook de exploración construye OrdersFull mediante JOIN sucesivos de las nueve tablas del dataset. El resultado es una tabla plana con aproximadamente 50 columnas que concentra toda la información de un pedido en una sola fila:

Grupo de columnas	Columnas representativas	Origen
Pedido	order_id, order_status, order_purchase_timestamp, order_delivered_customer_date	orders
Cliente	customer_id, customer_unique_id, customer_city, customer_state, customer_zip_code_prefix	customers
Ítem	order_item_id, shipping_limit_date, price, freight_value	order_items
Producto	product_id, product_category_name, product_weight_g, product_length_cm	products
Vendedor	seller_id, seller_city, seller_state, seller_zip_code_prefix	sellers
Pago	payment_sequential, payment_type, payment_installments, payment_value	order_payments
Reseña	review_id, review_score, review_comment_message, review_creation_date	order_reviews
Categoría	product_category_name_english	product_category

Tabla 1. Grupos de columnas en OrdersFull

1.2 Violaciones de Primera Forma Normal (1FN)

Una relación cumple 1FN cuando todos sus atributos son atómicos (indivisibles) y no existen grupos repetitivos. En OrdersFull se identifican las siguientes violaciones:

Violación	Columnas afectadas	Descripción
Grupos repetitivos de ítems	order_item_id, price, freight_value, product_id, seller_id	Un pedido puede tener múltiples ítems; en el JOIN cada ítem genera una fila duplicando todos los datos del pedido.
Grupos repetitivos de pagos	payment_sequential, payment_type, payment_value	Un pedido puede pagarse con varios métodos, generando filas adicionales con datos de pedido duplicados.
Grupos repetitivos de reseñas	review_id, review_score, review_comment_message	Aunque normalmente hay una reseña por pedido, el JOIN puede generarlas duplicadas al cruzar con pagos e ítems.

Tabla 2. Violaciones de 1FN en OrdersFull

Solución aplicada: separar orders, order_items, order_payments y order_reviews en tablas independientes. Cada tabla tiene una clave primaria bien definida y no contiene grupos repetitivos.

1.3 Violaciones de Segunda Forma Normal (2FN)

Una relación cumple 2FN si está en 1FN y cada atributo no clave depende de la totalidad de la clave primaria. En order_items la PK es compuesta (order_id, order_item_id). Se identifican las siguientes dependencias parciales:

Atributo	Depende solo de...	Violación
product_category_name, product_weight_g, product_length_cm	product_id	Las características del producto dependen solo del producto, no del ítem del pedido.
seller_city, seller_state	seller_id	Los datos del vendedor dependen solo del vendedor, no del ítem del pedido.
customer_city, customer_state	customer_id	Los datos del cliente dependen solo del cliente, no del pedido.

Tabla 3. Violaciones de 2FN — dependencias parciales

Solución aplicada: extraer products, sellers y customers como tablas independientes, referenciadas por FK desde order_items y orders respectivamente.

1.4 Violaciones de Tercera Forma Normal (3FN)

Una relación cumple 3FN si está en 2FN y no existen dependencias transitivas (atributo no clave que depende de otro atributo no clave). Se identifican dos dependencias transitivas significativas:

Tabla	$A \rightarrow B \rightarrow C$ (transitiva)	Dependencia transitiva	Solución
customers	customer_id $\rightarrow$ zip_code_prefix $\rightarrow$ city, state	city y state dependen del zip, no del customer_id	Eliminar city/state de customers; obtener via JOIN con geolocation
sellers	seller_id $\rightarrow$ zip_code_prefix $\rightarrow$ city, state	city y state dependen del zip, no del seller_id	Eliminar city/state de sellers; obtener via JOIN con geolocation
products	product_id $\rightarrow$ product_category_name $\rightarrow$ category_name_english	La traducción depende de la categoría, no del producto	Tabla product_category ya separada correctamente

Tabla 4. Violaciones de 3FN — dependencias transitivas

1.5 Esquema Normalizado Final

Tras aplicar 1FN, 2FN y 3FN, el esquema queda compuesto por nueve tablas. El siguiente diagrama muestra el modelo normalizado con las correcciones aplicadas:

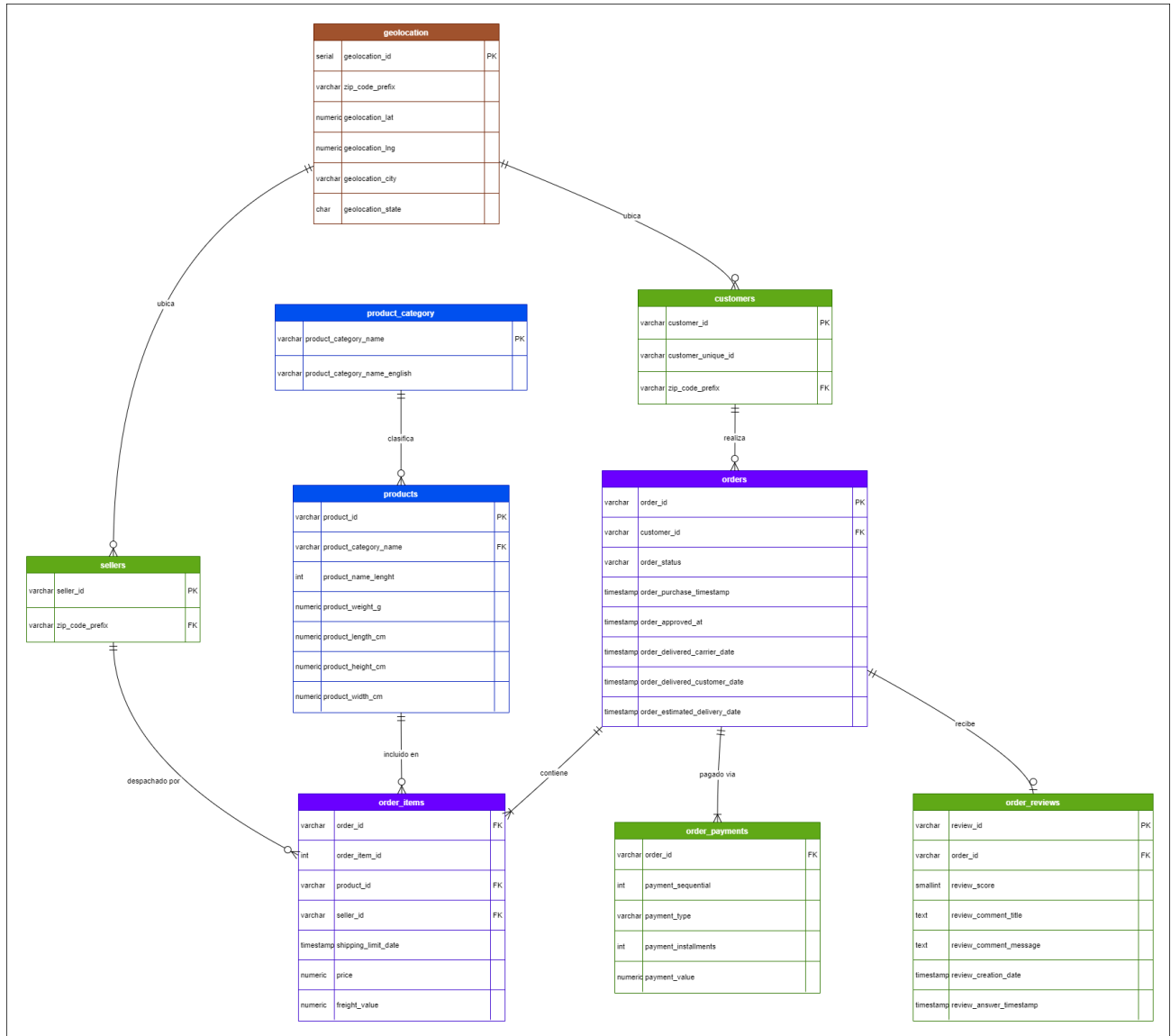


Figura 1. Diagrama ER normalizado — campos en rojo deben eliminarse (3FN)

Claves primarias y foráneas por tabla

Tabla	PK	FK(s)
product_category	product_category_name	—
geolocation	geolocation_id (surrogate)	—
customers	customer_id	zip_code_prefix → geolocation
sellers	seller_id	zip_code_prefix → geolocation
products	product_id	product_category_name → product_category
orders	order_id	customer_id → customers
order_items	(order_id, order_item_id)	order_id → orders, product_id → products, seller_id → sellers
order_payments	(order_id, payment_sequential)	order_id → orders
order_reviews	review_id	order_id → orders

Tabla 5. Claves primarias y foráneas del esquema normalizado

1.6 Análisis de Trade-offs

Ventajas de la alta normalización

- Eliminación de redundancia: cada dato se almacena una sola vez, reduciendo el espacio en disco y el riesgo de inconsistencias.
- Integridad referencial garantizada: las FK aseguran que no existan pedidos sin cliente, ni ítems sin producto.
- Facilidad de actualización: cambiar el nombre de una ciudad requiere actualizar solo la tabla geolocation, no miles de filas en customers.
- Flexibilidad para consultas analíticas: el modelo normalizado permite agregar desde cualquier dimensión sin datos duplicados.
- Menor anomalía de inserción: se puede registrar un producto sin que exista aún ningún pedido que lo referencie.

Desventajas de la alta normalización

- Mayor complejidad de consultas: obtener el nombre de la ciudad de un cliente requiere JOIN con geolocation.
- Rendimiento en lecturas frecuentes: consultas analíticas que cruzan muchas tablas pueden ser lentas sin índices adecuados.

- Dificultad de mantenimiento: más tablas implican mayor complejidad en el modelo y en el código de aplicación.
- Latencia en reportes: dashboards que necesitan datos de 5-6 tablas simultáneamente incurrir en costo de JOIN.

1.7 Desnormalización Estratégica para Ecommify

Tabla / módulo	Decisión	Justificación	Estrategia
orders + customers	Normalizar hasta 3FN	Datos transaccionales; alta frecuencia de escritura y actualización	PostgreSQL con FK estrictas
order_items + products	Normalizar hasta 3FN	Integridad crítica; un precio mal replicado afecta contabilidad	PostgreSQL + precio snapshot en order_items
order_payments	Normalizar hasta 3FN	Auditoría financiera exige trazabilidad exacta	PostgreSQL con CHECK y NOT NULL estrictos
order_reviews	Desnormalizar parcialmente	Solo lectura intensiva; el score y comment se consultan juntos siempre	Agregar columna review_score_avg a orders para dashboards
Catálogo de productos	Desnormalizar en caché	Alta lectura, baja escritura; nombre de categoría en inglés se consulta con cada producto	Materializar product + category en MongoDB para el frontend
Geolocalización analítica	Desnormalizar en data warehouse	Para reportes geográficos, el JOIN con geolocation es costoso	Vista materializada o tabla denormalizada en capa analítica

Tabla 6. Decisiones de normalización vs. desnormalización por módulo

2. Modelado Conceptual, Lógico y Físico

El diseño de una base de datos atraviesa tres fases iterativas: conceptual (qué entidades y relaciones existen en el dominio), lógica (cómo se representan esas entidades en tablas relacionales) y física (cómo se implementan eficientemente en un motor específico, PostgreSQL en este caso).

2.1 Fase Conceptual

El modelo conceptual captura el dominio del negocio independientemente de la tecnología. Se identifican las entidades, sus atributos principales y las relaciones entre ellas.

Entidades y atributos del dominio

Entidad	Atributos principales	Descripción del dominio
CLIENTE	id_cliente, id_unico, ciudad, estado, codigo_postal	Comprador registrado en la plataforma. Un cliente puede tener múltiples pedidos.
PEDIDO	id_pedido, estado, fecha_compra, fecha_entrega, fecha_estimada	Unidad central de transacción. Tiene un ciclo de vida desde la compra hasta la entrega.
PRODUCTO	id_producto, nombre, peso, dimensiones, fotos	Artículo disponible en el catálogo. Pertenece a una categoría.
VENDEDOR	id_vendedor, ciudad, estado, codigo_postal	Comercio que ofrece productos en el marketplace.
PAGO	secuencial, tipo, cuotas, valor	Registro de cada transacción financiera asociada a un pedido.
RESENA	id_resena, puntuacion, titulo, mensaje, fecha	Evaluación del comprador sobre el pedido recibido. Escala 1-5.
CATEGORIA	nombre_categoria, nombre_ingles	Clasificación jerárquica de productos. Incluye traducción al inglés.
GEOLOCALIZACION	codigo_postal, latitud, longitud, ciudad, estado	Referencia geográfica compartida por clientes y vendedores.

Tabla 7. Entidades del dominio Ecommify

Relaciones y cardinalidades

Entidad A	Cardinalidad	Entidad B	Semántica
CLIENTE	1 : N	PEDIDO	Un cliente realiza muchos pedidos; cada pedido pertenece a un único cliente.
PEDIDO	1 : N	ITEM_PEDIDO	Un pedido contiene uno o más ítems; cada ítem pertenece a un pedido.
PRODUCTO	N : M → ITEM_PEDIDO	PEDIDO	Un producto puede aparecer en muchos pedidos; un pedido puede incluir muchos productos.
VENDEDOR	1 : N	ITEM_PEDIDO	Un vendedor despacha muchos ítems; cada ítem tiene un vendedor responsable.
PEDIDO	1 : N	PAGO	Un pedido puede pagarse con múltiples métodos o en cuotas.
PEDIDO	1 : 0..1	RESEÑA	Un pedido puede recibir a lo sumo una reseña; la reseña es opcional.
CATEGORIA	1 : N	PRODUCTO	Una categoría agrupa muchos productos; cada producto pertenece a una categoría.
GEOLOCALIZACION	1 : N	CLIENTE / VENDEDOR	Un código postal ubica a múltiples clientes y vendedores.

Tabla 8. Relaciones y cardinalidades del modelo conceptual

Diagrama ER conceptual

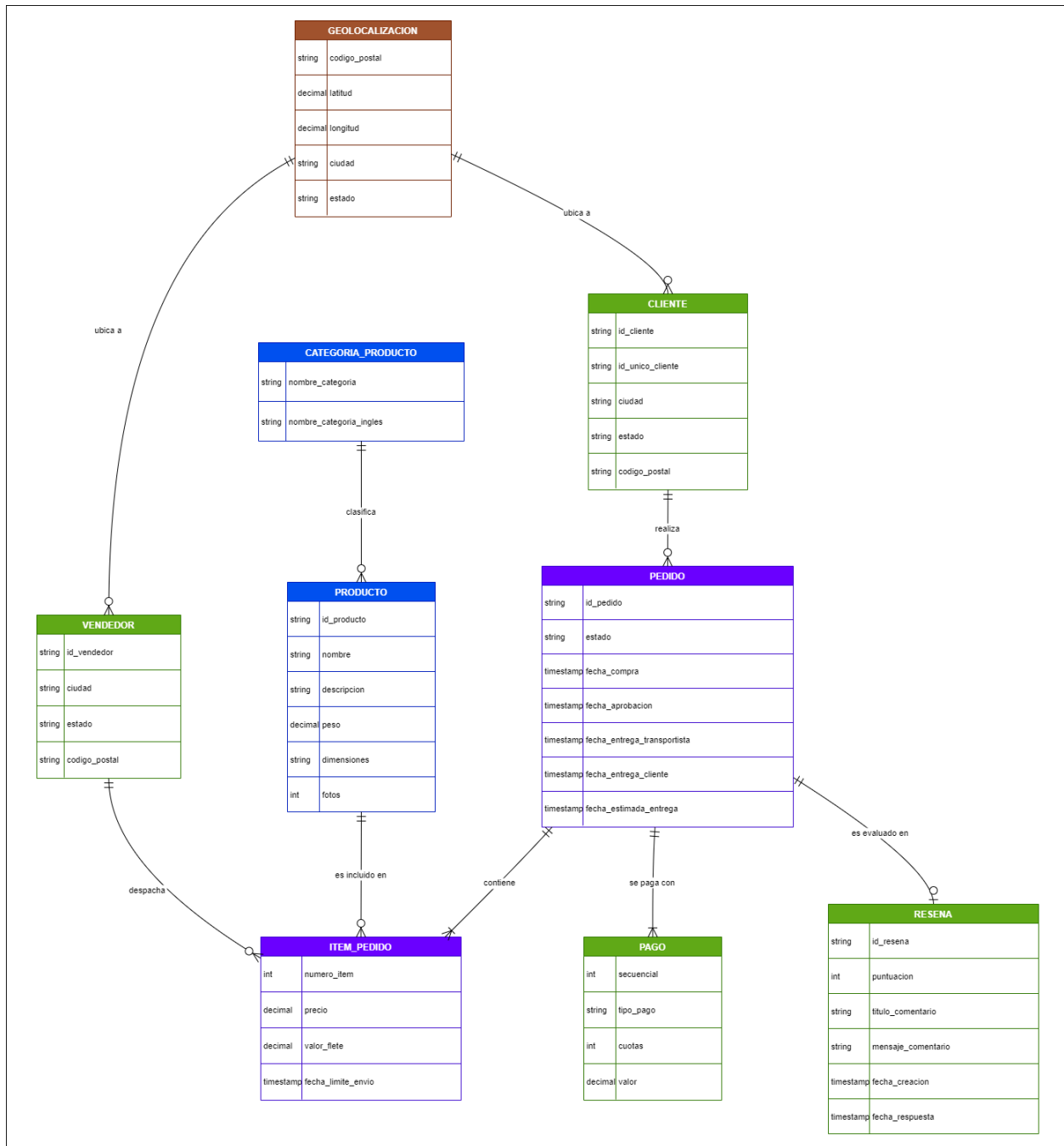


Figura 2. Diagrama ER conceptual (estilo Crow's Foot) — dataset Olist / Ecommify

2.2 Fase Lógica

La fase lógica transforma el modelo conceptual en un esquema relacional aplicando reglas formales de conversión.

Reglas de transformación aplicadas

- Entidad → tabla: cada entidad del modelo conceptual se convierte en una tabla relacional.
- Relación 1:N → FK: la FK se coloca en el lado "muchos" (e.g., customer_id en orders).
- Relación N:M → tabla intermedia: la relación entre PRODUCTO y PEDIDO genera order_items con FK a ambas entidades más atributos propios de la relación (price, freight_value).
- Entidad débil: ITEM_PEDIDO usa PK compuesta (order_id, order_item_id) porque su existencia depende de PEDIDO.
- Atributos multivaluados: los pagos múltiples se resuelven con la tabla order_payments con PK compuesta (order_id, payment_sequential).

Restricciones de integridad definidas

Tabla	Tipo de restricción	Definición
order_reviews	CHECK	review_score BETWEEN 1 AND 5 — garantiza escala válida de calificación.
order_payments	CHECK	payment_value > 0 — ningún pago puede ser de valor negativo o cero.
order_items	CHECK	price >= 0 AND freight_value >= 0 — valores monetarios no negativos.
orders	CHECK	order_delivered_customer_date >= order_purchase_timestamp — la entrega no puede ser anterior a la compra.
customers	UNIQUE	customer_unique_id — identifica compradores reales entre pedidos distintos.
orders	NOT NULL	customer_id, order_status, order_purchase_timestamp — campos obligatorios del ciclo de vida.
order_items	NOT NULL	order_id, product_id, seller_id, price, freight_value — ítem incompleto invalida el pedido.

Tabla 9. Restricciones de integridad del esquema lógico

2.3 Fase Física

La fase física concreta el esquema lógico en construcciones específicas de PostgreSQL, seleccionando tipos de datos, índices y estrategias de particionamiento.

Selección de tipos de datos en PostgreSQL

Columna representativa	Tipo PostgreSQL	Alternativa descartada	Justificación
order_id, customer_id, etc.	VARCHAR(50)	UUID	El dataset usa hashes MD5 de 32 chars; UUID añade overhead innecesario.
price, freight_value, payment_value	NUMERIC(10,2)	FLOAT	NUMERIC garantiza precisión decimal exacta; FLOAT introduce errores de redondeo en valores monetarios.
order_purchase_time stamp	TIMESTAMP	DATE	Se requiere hora exacta para análisis de patrones temporales intra-día.
review_score	SMALLINT	INT	Rango 1-5; SMALLINT ocupa 2 bytes vs 4 bytes de INT. Con millones de reseñas el ahorro es significativo.
customer_state, seller_state	CHAR(2)	VARCHAR	Longitud fija garantizada; CHAR es ligeramente más eficiente para valores de longitud constante.
review_comment_message	TEXT	VARCHAR(n)	Longitud variable ilimitada; los comentarios no tienen tamaño predecible.
geolocation_lat, geolocation_lng	NUMERIC(10,6)	FLOAT8	Precisión de 6 decimales (~11 cm de resolución) sin error de punto flotante.

Tabla 10. Selección de tipos de datos PostgreSQL

Plan de índices

Tabla	Índice	Justificación
orders	idx_orders_customer_id	Consulta frecuente: "todos los pedidos de un cliente".
orders	idx_orders_purchase_timestamp	Filtros por rango de fecha en reportes temporales y dashboards.
orders	idx_orders_status	Filtro por estado del pedido en operaciones de logística.
order_items	idx_order_items_product_id	Consulta frecuente: "todos los pedidos que incluyen un producto".
order_items	idx_order_items_seller_id	Panel del vendedor: sus ítems despachados.
order_reviews	idx_order_reviews_score	Filtros por puntuación en reportes de calidad.
products	idx_products_category	Navegación por categoría en el catálogo.
customers	idx_customers_unique_id	Identificar el mismo comprador entre múltiples customer_id.
geolocation	idx_geolocation_zip (B-tree)	JOIN con customers y sellers por zip_code_prefix.

Tabla 11. Plan de índices inicial para PostgreSQL

Consideraciones de particionamiento

Las tablas orders y order_items son las de mayor volumen (el dataset Olist tiene ~100,000 pedidos y ~112,000 ítems; en producción puede escalar a millones). Se propone particionamiento por rango de fecha:

```
-- Particionamiento por rango en orders
CREATE TABLE orders (
  order_id VARCHAR(50) NOT NULL,
  order_purchase_timestamp TIMESTAMP NOT NULL,
  ...
) PARTITION BY RANGE (order_purchase_timestamp);

CREATE TABLE orders_2016 PARTITION OF orders
  FOR VALUES FROM ('2016-01-01') TO ('2017-01-01');

CREATE TABLE orders_2017 PARTITION OF orders
  FOR VALUES FROM ('2017-01-01') TO ('2018-01-01');

CREATE TABLE orders_2018 PARTITION OF orders
  FOR VALUES FROM ('2018-01-01') TO ('2019-01-01');
```

Ventajas del particionamiento: las consultas por rango de fecha solo escanean la partición relevante, reduciendo significativamente el I/O. Las particiones antiguas pueden archivarse o moverse a tablespaces más lentos.

2.4 Decisiones de diseño justificadas

Decisión	Alternativa considerada	Justificación técnica
geolocation_id SERIAL como PK surrogate	zip_code_prefix como PK natural	El dataset tiene múltiples filas con el mismo zip pero coords distintas; la PK natural no sería única.
price almacenado en order_items (snapshot)	Solo referenciar products.price	El precio en el momento de la compra puede diferir del precio actual del producto. El snapshot garantiza auditoría correcta.
customer_id ≠ customer_unique_id	Usar un solo ID	El dataset usa customer_id por pedido (permite rastrear el pedido exacto) y customer_unique_id para identificar al comprador real entre varios pedidos.
order_reviews opcionales (0..1)	Obligatorias (1..1)	No todos los pedidos reciben reseña; forzarla generaría registros vacíos o bloquearía el cierre del pedido.

Tabla 12. Decisiones de diseño y justificación

3. Comparación de Paradigmas: Relacional vs. NoSQL

La arquitectura de datos de Ecommify no requiere una única tecnología. El Teorema CAP establece que ningún sistema distribuido puede garantizar simultáneamente Consistencia (C), Disponibilidad (A) y Tolerancia a particiones (P). Esta sección analiza cada módulo del sistema para seleccionar el paradigma y tecnología más adecuados.

3.1 Análisis por módulo según Teorema CAP

Módulo	Prioridad CAP	Datos	Patrón acceso	Razonamiento
Pedidos y clientes	CP (Consistencia)	Estructurados	Transaccional OLTP	Un pedido no puede existir sin cliente. La integridad referencial es crítica; no se acepta disponibilidad eventual.
Pagos	CP (Consistencia)	Estructurados	Transaccional OLTP	Las transacciones financieras exigen ACID estricto. Una doble escritura o pérdida de registro es inaceptable.
Inventario / Ítems	CP (Consistencia)	Estructurados	Transaccional OLTP	El stock y los precios en el momento de compra deben ser precisos. Los JOINS con productos y vendedores son frecuentes.
Catálogo de productos	AP (Disponibilidad)	Semi-estructurado	Lectura intensiva	Los atributos de producto varían por categoría (electrónicos vs. ropa). Alta lectura, baja escritura. Consistencia eventual aceptable.
Reseñas y comentarios	AP (Disponibilidad)	Semi-estructurado	Lectura intensiva	El usuario puede ver una reseña levemente desactualizada sin impacto crítico. Estructura variable (título, tags, fotos).
Geolocalización analítica	AP (Disponibilidad)	Estructurado	Analítico OLAP	Consultas de mapas y reportes geográficos; no requieren consistencia transaccional. Latencia tolerable.
Sesiones y carrito	AP (Disponibilidad)	Semi-estructurado	Alta escritura/lectura	El carrito de compra es volátil y de estructura variable. La pérdida temporal es tolerable; la disponibilidad es prioritaria.

Tabla 13. Análisis CAP por módulo de Ecommify

3.2 Matriz de decisión PostgreSQL vs. MongoDB

Módulo / entidad	Tecnología	Modelo datos	ACID	Justificación técnica
orders, customers, sellers	PostgreSQL	Relacional 3FN	Completo	Integridad referencial obligatoria; transacciones ACID para el ciclo de vida del pedido.
order_items, order_payments	PostgreSQL	Relacional 3FN	Completo	Datos financieros; no se acepta consistencia eventual. FK garantizan integridad con orders y products.
products (catálogo frontend)	MongoDB	Documento JSON	No requerido	Atributos variables por categoría (un zapato tiene talla, un libro tiene ISBN). Esquema flexible sin migraciones DDL.
order_reviews	MongoDB	Documento JSON	No requerido	Comentarios con estructura variable (fotos adjuntas, tags, longitud libre). Alta lectura; consistencia eventual aceptable.
Carrito de compras	MongoDB	Documento JSON	No requerido	Estado efímero y de alta frecuencia. TTL index de MongoDB simplifica la expiración automática del carrito.
Geolocalización (reportes)	PostgreSQL + PostGIS	Relacional + geo	No requerido	PostGIS agrega tipos geoespaciales nativos; permite consultas de distancia y polígonos sin ETL adicional.
Analytics / OLAP	PostgreSQL (particionado)	Relacional	No requerido	Las consultas analíticas se aíslan del OLTP mediante particionamiento por fecha y réplica de solo lectura.

Tabla 14. Matriz de decisión relacional vs. NoSQL por módulo

3.3 Arquitectura híbrida propuesta

La arquitectura combina PostgreSQL como sistema de registro (source of truth para datos transaccionales) con MongoDB para datos de alta lectura y esquema flexible. La sincronización entre ambos se realiza mediante eventos de dominio:

Capa	Componentes
Capa transaccional (PostgreSQL)	orders, customers, sellers, order_items, order_payments — escritura ACID, FK estrictas, particionamiento por fecha.
Capa de catálogo (MongoDB)	products (colección con subdocumentos de atributos por categoría), order_reviews (documentos con campos opcionales).
Capa de sesión (MongoDB)	Colección carts con TTL index para expiración automática. Alta frecuencia de escritura/lectura.
Capa analítica (PostgreSQL réplica)	Réplica de solo lectura con vistas materializadas para dashboards. Particionamiento por año en orders y order_items.
Capa geoespacial (PostGIS)	Extensión de PostgreSQL con índice GIST en coordenadas. Consultas de radio, heatmaps y reportes por estado.
Sincronización	Change Data Capture (CDC) con Debezium sobre PostgreSQL → Kafka → MongoDB. Los cambios en products de PG se propagan al catálogo Mongo en < 1 segundo.

Tabla 15. Capas de la arquitectura híbrida propuesta

3.4 Estrategia de sincronización

El riesgo principal de una arquitectura híbrida es la inconsistencia entre sistemas. Se propone la siguiente estrategia:

1. Change Data Capture (CDC): Debezium monitorea el WAL (Write-Ahead Log) de PostgreSQL y publica cada cambio como evento en Kafka. MongoDB consume estos eventos de forma asíncrona.
2. Tolerancia a inconsistencia temporal: el catálogo de MongoDB puede estar desactualizado hasta 1 segundo. Esto es aceptable para lecturas de frontend pero no para transacciones de compra (que siempre consultan PostgreSQL).
3. Identificador compartido: product_id es la clave que vincula el documento MongoDB con la fila PostgreSQL. Nunca se genera en MongoDB; siempre en PostgreSQL primero.
4. Manejo de fallos: si Kafka cae, PostgreSQL sigue operando normalmente (es el sistema primario). MongoDB puede quedar desactualizado temporalmente; se reconcilia al restaurar el stream.
5. Consistencia eventual verificable: un job diario compara los conteos y hashes de product_id entre ambos sistemas y reporta desviaciones.

3.5 Comparación técnica consolidada

Criterio	PostgreSQL	MongoDB
Modelo de datos	Tablas con esquema fijo y tipos estrictos	Documentos JSON con esquema flexible por colección
Transacciones	ACID completo, incluyendo transacciones distribuidas	ACID en documento único; transacciones multi-documento desde v4.0
Escalabilidad	Vertical principalmente; sharding posible pero complejo	Sharding nativo y horizontal; diseñado para escalar out
Consultas	SQL estándar; JOIN optimizados con índices	MQL (MongoDB Query Language); \$lookup para joins (menos eficiente)
Esquema variable	Requiere ALTER TABLE y migraciones	Soportado nativamente; diferentes documentos pueden tener campos distintos
Geoespacial	PostGIS: soporte completo de estándares OGC	Índices 2dsphere: consultas de proximidad y polígonos
Mejor para Ecommify	Transacciones, pagos, inventario, auditoría	Catálogo, reseñas, carrito, notificaciones

Tabla 16. Comparación técnica PostgreSQL vs. MongoDB para Ecommify

Referencias

- Brewer, E. A. (2000). Towards robust distributed systems. Proceedings of the 19th Annual ACM Symposium on Principles of Distributed Computing (PODC), 7.
<https://doi.org/10.1145/343477.343502>
- Codd, E. F. (1970). A relational model of data for large shared data banks. Communications of the ACM, 13(6), 377–387. <https://doi.org/10.1145/362384.362685>
- Date, C. J. (2003). An introduction to database systems (8th ed.). Addison-Wesley.
- Kleppmann, M. (2017). Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems. O'Reilly Media.
- MongoDB, Inc. (2024). MongoDB manual. <https://www.mongodb.com/docs/manual/>
- Olist. (2018). Brazilian e-commerce public dataset by Olist [Dataset]. Kaggle.
<https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>
- PostgreSQL Global Development Group. (2024). PostgreSQL 16 documentation.
<https://www.postgresql.org/docs/>
- Stonebraker, M., & Cattell, R. (2011). 10 rules for scalable performance in simple operations databases. Communications of the ACM, 54(6), 72–80.
<https://doi.org/10.1145/1953122.1953144>